

Afrobotics BIT — System Design Document

Document Version: 1.0
Date: 20 July 2026
Author: Afrobotics Engineering
Status: Draft
Classification: Confidential

Table of Contents

- 1. [Document Information](#)
 - 2. [Executive Summary](#)
 - 3. [System Overview & Goals](#)
 - 4. [Architecture Overview](#)
 - 5. [Technology Stack](#)
 - 6. [Domain Model](#)
 - 7. [Subsystem Design](#)
 - 8. [AI / ML Pipeline Design](#)
 - 9. [API Design](#)
 - 10. [Frontend Architecture](#)
 - 11. [Security Architecture](#)
 - 12. [Data Flow & Integration Patterns](#)
 - 13. [Deployment Architecture](#)
 - 14. [Monitoring & Observability](#)
 - 15. [Non-Functional Requirements](#)
 - 16. [Risks, Assumptions & Mitigations](#)
 - 17. [Appendix](#)
-

1. Document Information

Field	Value
Project Name	Afrobotics Brand Insertion Technology (BIT) — Dynamic Virtual Product Placement & In-Content Advertising Platform
Document Purpose	Technical design specification describing system architecture, components, interfaces, AI pipeline, data model, and implementation strategy
Intended Audience	Engineering team, technical architects, QA, DevOps, stakeholders
Reference Documents	REQUIREMENT_SPECIFICATION.md (MReq 1–25), Functional Specification, Testing Manual
Revision	1.0 (Initial Design)

2. Executive Summary

Afrobotics Brand Insertion Technology (BIT) is an **AI-powered video inventory creation platform** that enables brands to be inserted into existing video content after production. It transforms suitable surfaces inside video — billboards, screens, walls, signage, product spaces — into monetisable advertising inventory while appearing visually indistinguishable from the original scene.

The platform targets African broadcasters, streaming platforms (SVOD/AVOD), and production houses, creating a new **non-interruptive advertising layer** inside content that currently generates zero revenue from its internal visual real estate.

2.1 Key Capabilities

- **Content Ingestion** — Accept video, transcode, detect scenes, extract metadata
- **AI Scene Analysis** — Computer-vision surface detection, depth estimation, orientation mapping
- **Placement Recommendation** — Score and rank candidate surfaces; brand-safety enforcement
- **Motion Tracking** — Lock brand assets to surfaces through camera movement
- **AI Compositing** — Perspective warp, relight, blur, shadow matching for photo-realism
- **GPU Rendering** — Batch render broadcast/streaming/social output
- **Campaign Management** — Advertiser campaigns, asset library, scheduling, regional targeting
- **Approval Workflow** — Mandatory human-in-the-loop approval; audit trail
- **Analytics & BI** — Exposure metrics, performance reporting, real-time dashboards
- **Enterprise Operations** — RBAC, JWT auth, event logging, alarms, redundancy

3. System Overview & Goals

3.1 Vision

"Turn every surface in existing premium video content into a monetisable, brand-safe advertising opportunity — without disrupting the viewer experience."

3.2 Core Principles

1. **AI-First, Human-Guarded** — AI automates detection, tracking, and compositing; humans approve every placement (MReq 4, 11).
2. **Brand Safety is Non-Negotiable** — Permanent exclusion list enforced at placement time; configurable but never silently reduced (MReq 4).
3. **Modular & Swappable** — Every AI component is interface-abstracted so the engine can be swapped without re-architecting.
4. **Content Integrity** — Duration and visual content of source footage are never altered beyond approved insertions (MReq 1).
5. **Cloud-Native** — Deploy on virtual infrastructure with GPU compute; scale horizontally.

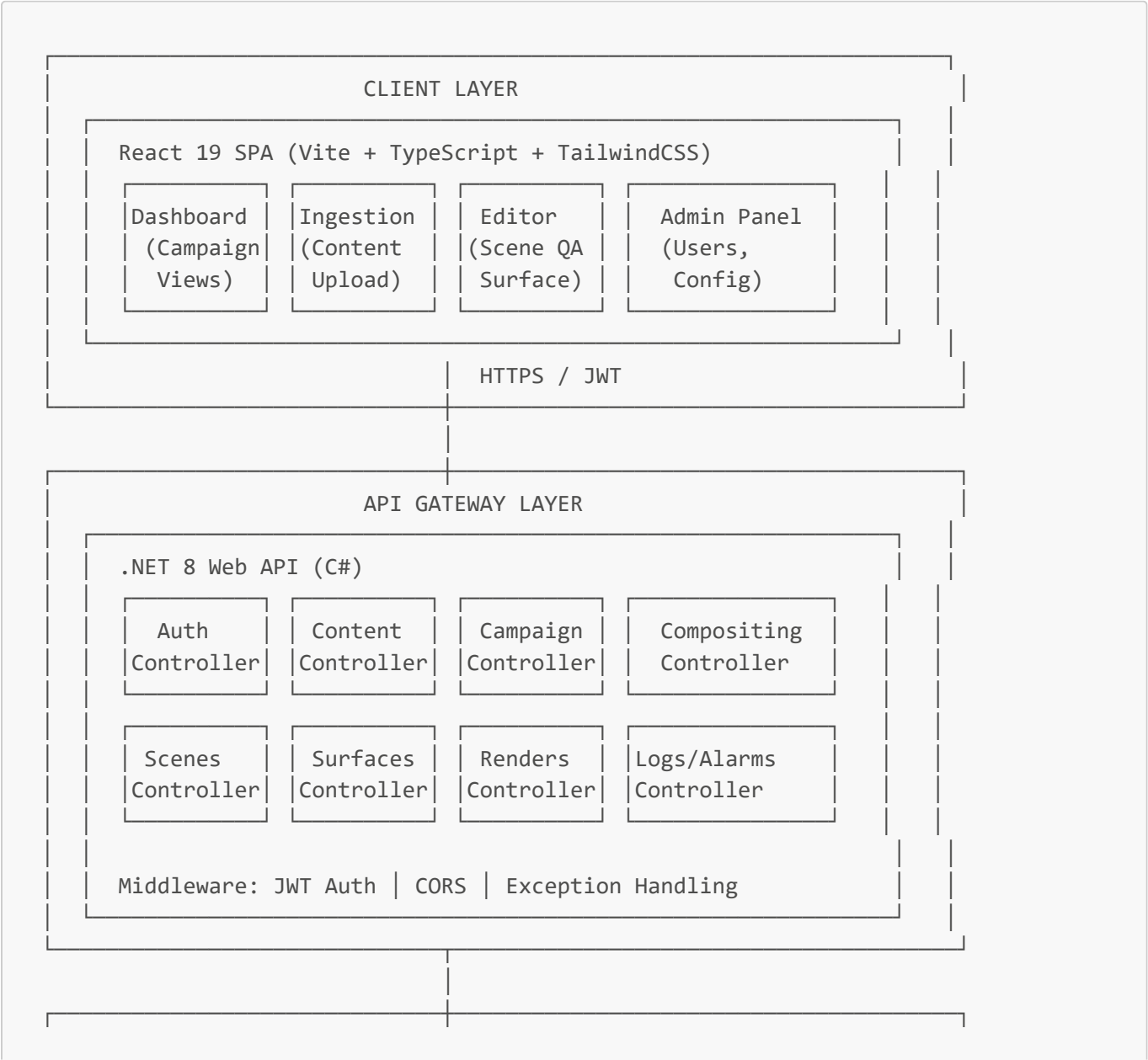
3.3 Main Subsystems

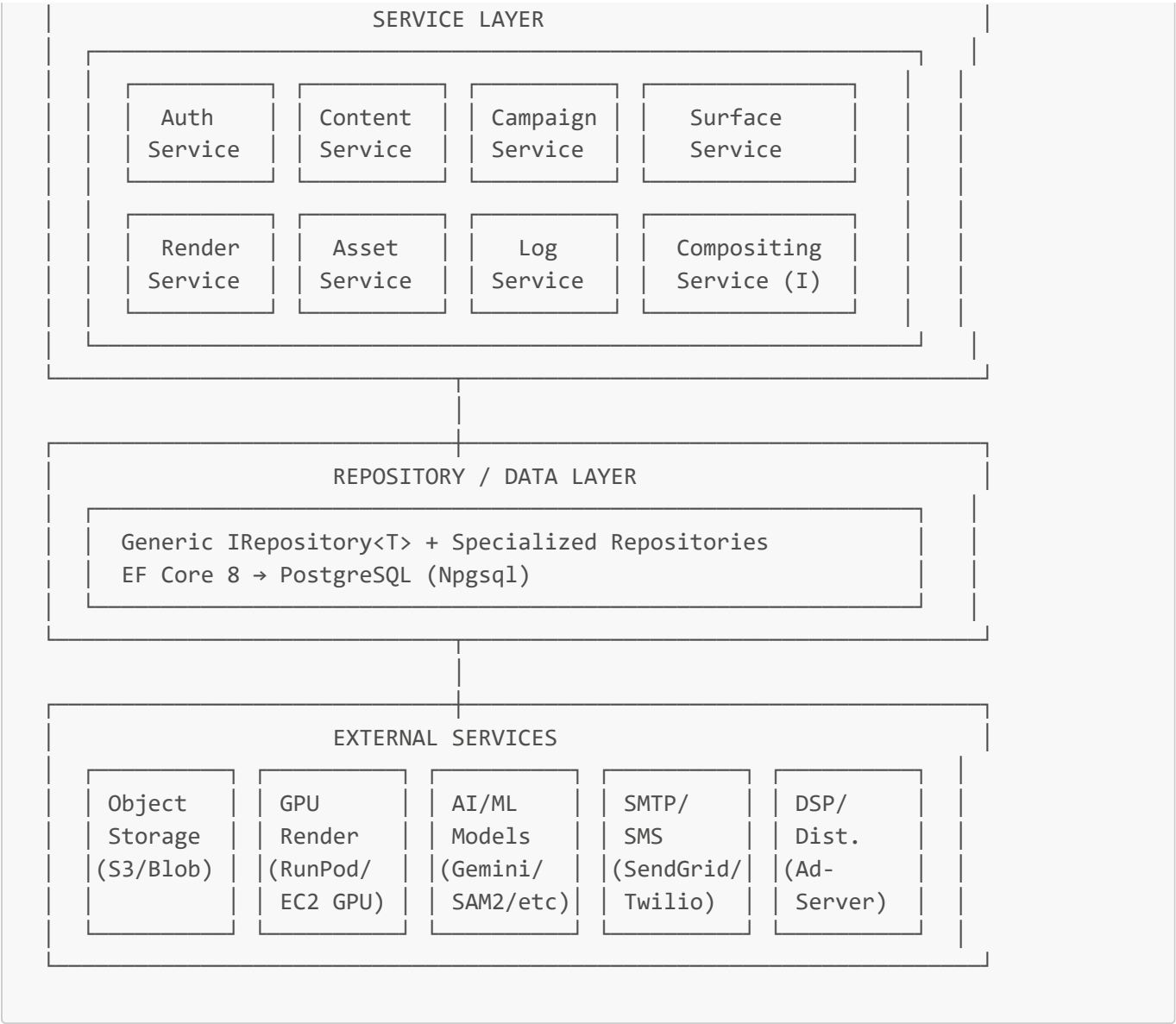
Subsystem	Responsibility
Ingestion	Accept source video, transcode, scene-cut detection, metadata extraction

Subsystem	Responsibility
Scene Analysis	Computer-vision surface detection, depth/orientation estimation
Placement Recommendation	Score & rank candidate surfaces; brand-safety filter
Motion Tracking	Planar/point tracking to lock assets across camera movement
Compositing & Rendering	Warp, relight, blur, shadow; GPU render to distributable formats
Campaign & Inventory	Advertiser campaigns, creative assets, scheduling, regional targeting
Approval Workflow	Human review/approval; audit trail; mandatory gate
Analytics & BI	Exposure metrics, performance reports, real-time dashboard
Operations	Event logging, alarms, usage records, admin console

4. Architecture Overview

4.1 High-Level Architecture Diagram





4.2 Architectural Patterns

Pattern	Usage
Layered Architecture	Controllers → Services → Repositories → Database
Repository Pattern	Generic <code>IRepository<T></code> with specialized interfaces per aggregate
Strategy Pattern	<code>ICompositingService</code> allows swapping AI compositing engines
Dependency Injection	All services registered in DI container (<code>Program.cs</code>)
JWT Token Auth	Stateless authentication via Bearer tokens
SPA with API Proxy	Vite dev proxy <code>/api</code> → .NET backend; same origin in production

5. Technology Stack

5.1 Current (Implemented)

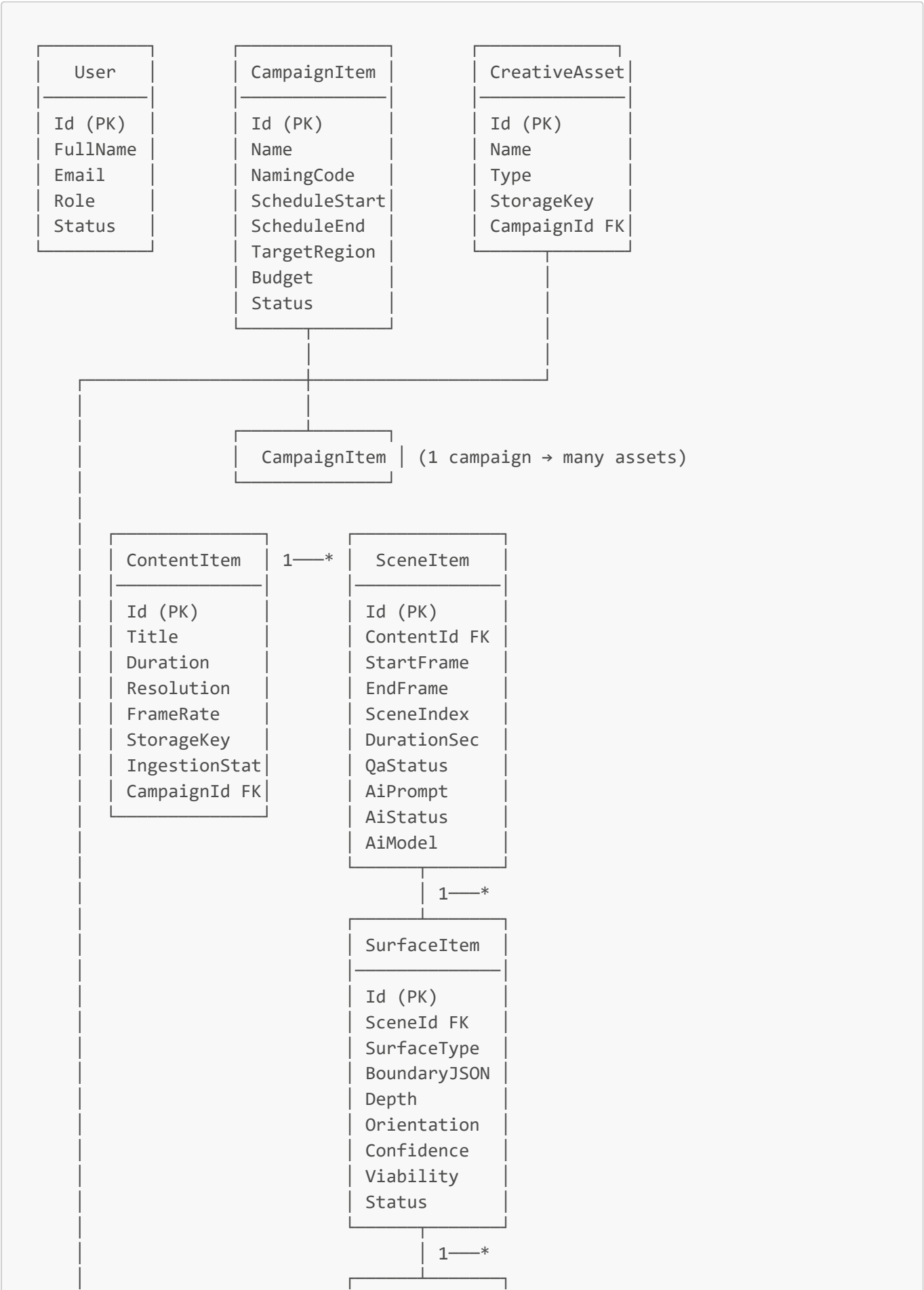
Layer	Technology	Version
Frontend Framework	React (with Hooks, Context)	19.0
Build Tool	Vite	6.2
Language	TypeScript	5.8
CSS Framework	TailwindCSS	4.1
Routing	React Router DOM	7.18
Animation	Motion (Framer Motion)	12.23
Icons	Lucide React	0.546
Backend Runtime	.NET (ASP.NET Core)	8.0
Language	C# 12	—
ORM	Entity Framework Core	8.0
Database	PostgreSQL (via Npgsql)	16+
Authentication	JWT Bearer (Microsoft.AspNetCore.Authentication.JwtBearer)	8.0
AI SDK (Client)	Google GenAI (@google/genai)	2.4

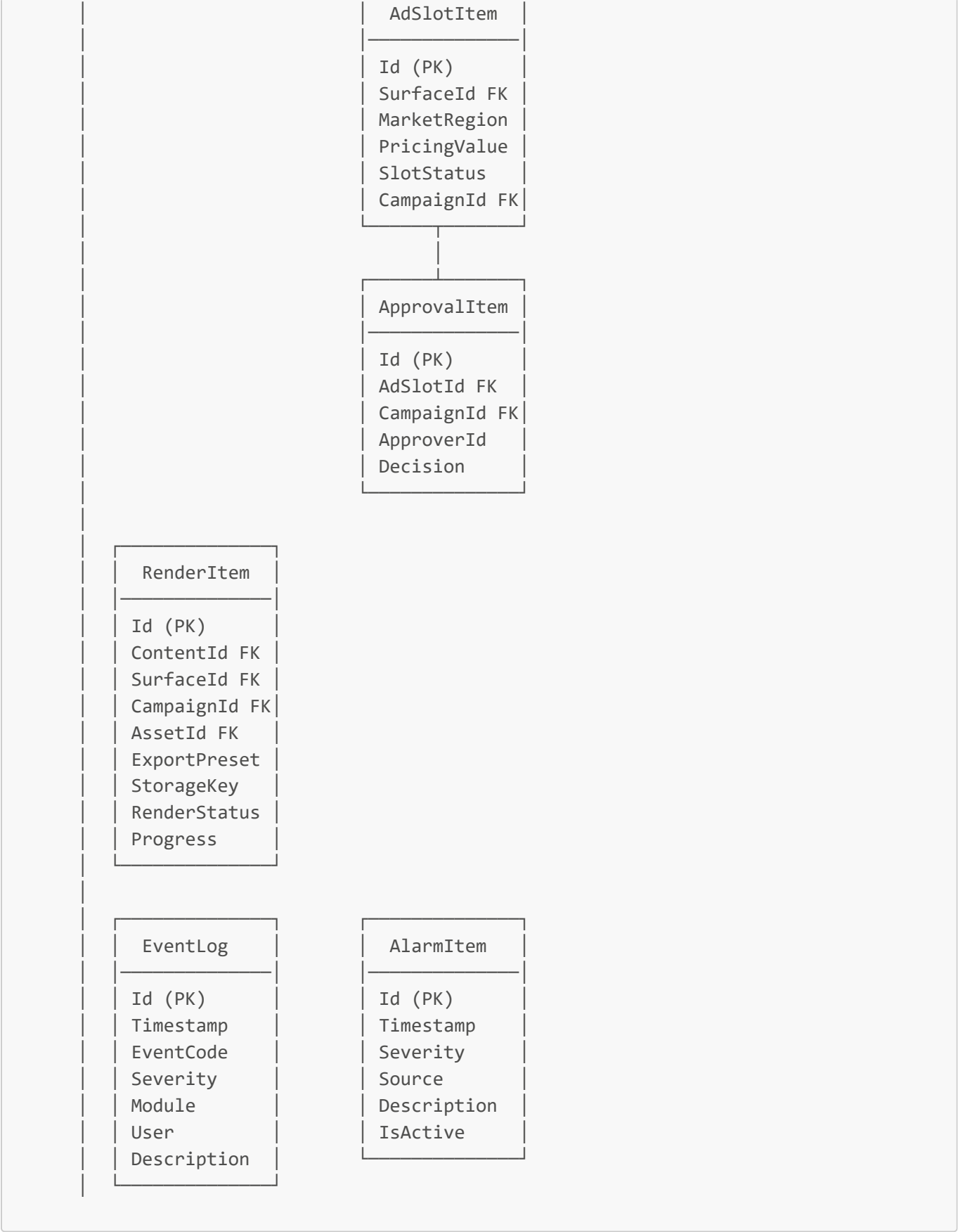
5.2 Planned / To Be Confirmed

Layer	Candidate Technologies	Status
Scene Detection	FFmpeg + custom scene-cut library	Build
Segmentation / Object Detection	SAM 2, YOLOv8, Detectron2	TBC (Wrap)
Motion Tracking	OpenCV planar tracker + point-tracking fallback	TBC (Wrap)
Compositing Engine	Runway Gen-4 API, SAM 2 + IC-Light, proprietary homography solver	TBC (Swap via ICompositingService)
GPU Rendering	RunPod, AWS EC2 GPU (G5 instances), FFmpeg, Blender	TBC
Object Storage	AWS S3, Azure Blob Storage, MinIO (on-prem)	TBC
Notifications	SendGrid/Mailgun (SMTP), Twilio/Africa's Talking (SMS)	TBC
Message Queue	RabbitMQ, Azure Service Bus, AWS SQS	TBC (for async job dispatch)
Containerization	Docker, Kubernetes (AKS/EKS)	Planned
CI/CD	GitHub Actions, Azure DevOps	Existing (GitHub)

6. Domain Model

6.1 Entity Relationship Diagram





6.2 Entity Summary

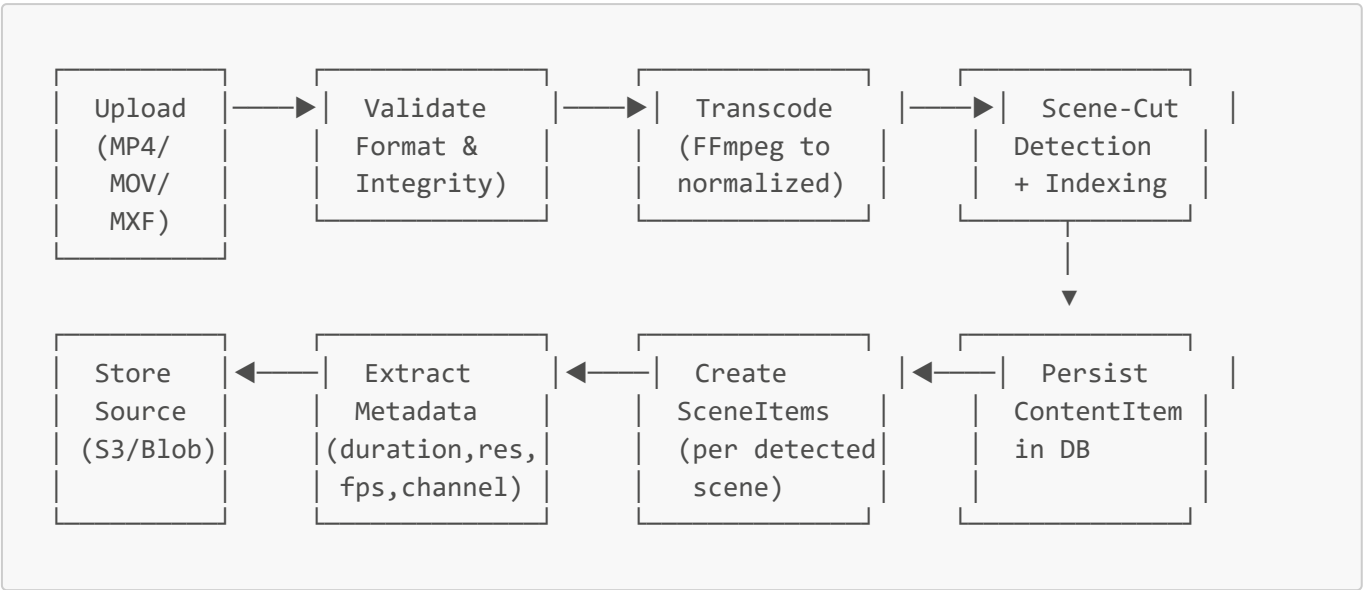
Entity	Purpose	Key Relationships
User	Platform user with role-based access	—

Entity	Purpose	Key Relationships
ContentItem	Ingested source video	Parent of SceneItems; optionally linked to Campaign
SceneItem	Detected scene segment within content	Child of ContentItem; parent of SurfaceItems
SurfaceItem	Candidate advertising surface in a scene	Child of SceneItem; parent of AdSlotItems
AdSlotItem	Approved, monetisable placement	Links Surface → Campaign; triggers Approvals, Renders
CampaignItem	Advertiser campaign definition	Parent of CreativeAssets, AdSlots, Renders
CreativeAsset	Brand asset (image, logo, video)	Belongs to Campaign
ApprovalItem	Human approval/rejection record	Links AdSlot, Campaign, Approver
RenderItem	Rendered output file	Links Content, Surface, Campaign, Asset
EventLog	System event record	—
AlarmItem	Active/cleared alarm	—

7. Subsystem Design

7.1 Ingestion Subsystem

Purpose: Accept source video, transcode to normalized format, extract metadata, perform scene-cut detection.



States: Staging → Transcoding → SceneDetecting → Completed | Failed

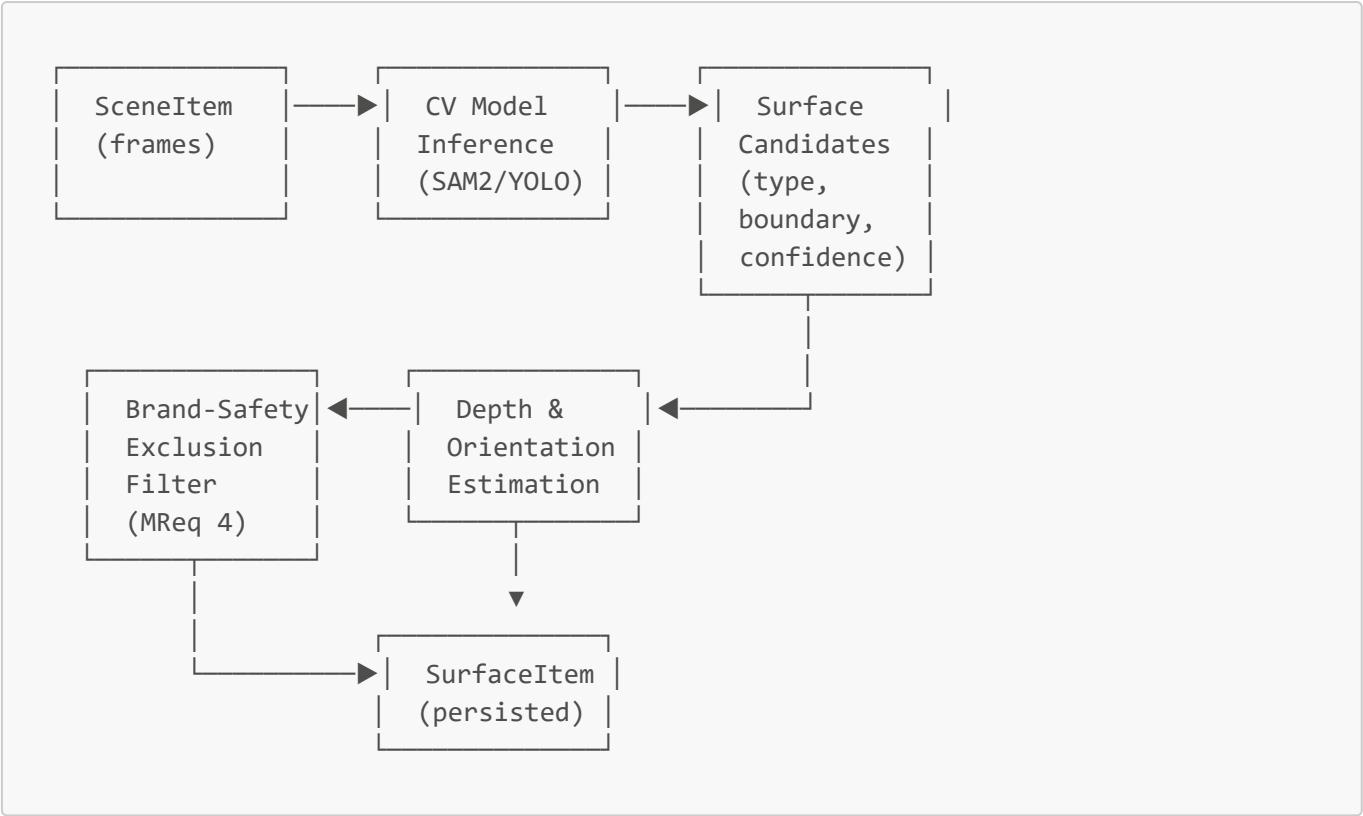
Key Constraints:

- Duration must not be modified from source (MReq 1)

- Strict campaign naming structure enforced: `XX00XX00_XXXX` `XX00XX00_XXXX` (MReq 1)
- Invalid/unsupported uploads rejected with clear error (MReq 1)

7.2 Scene Analysis Subsystem

Purpose: Run computer-vision models to detect candidate advertising surfaces per frame, estimate depth and orientation.



Detectable Surface Types:

- Billboards, screens (TV/LED), walls, signage, posters
- Stadium perimeter LED boards, product packaging, product spaces
- **Excluded (Permanent):** Human faces, children, emergency vehicles, government insignia, religious symbols/spaces

AI Models (TBC):

Function	Candidate	Rationale
Object detection	YOLOv8 / Detectron2	Proven, fast, well-documented
Segmentation	SAM 2 (Meta)	State-of-the-art, zero-shot capability
Depth estimation	MiDaS / Depth Anything v2	Monocular depth, good generalization
Orientation	Custom geometric solver	Homography-based from detected planes
Brand detection	Gemini Vision / custom classifier	Logo & text recognition for competitive separation

7.3 Placement Recommendation Subsystem

Purpose: Score and rank candidate surfaces; enforce brand-safety; present recommendations for human review.

Scoring Formula (conceptual):

$$Score = w_1 \cdot C_{confidence} + w_2 \cdot V_{visibility} + w_3 \cdot D_{duration} + w_4 \cdot S_{size} - w_5 \cdot O_{occlusion} - P_{brand_conflict}$$

Where:

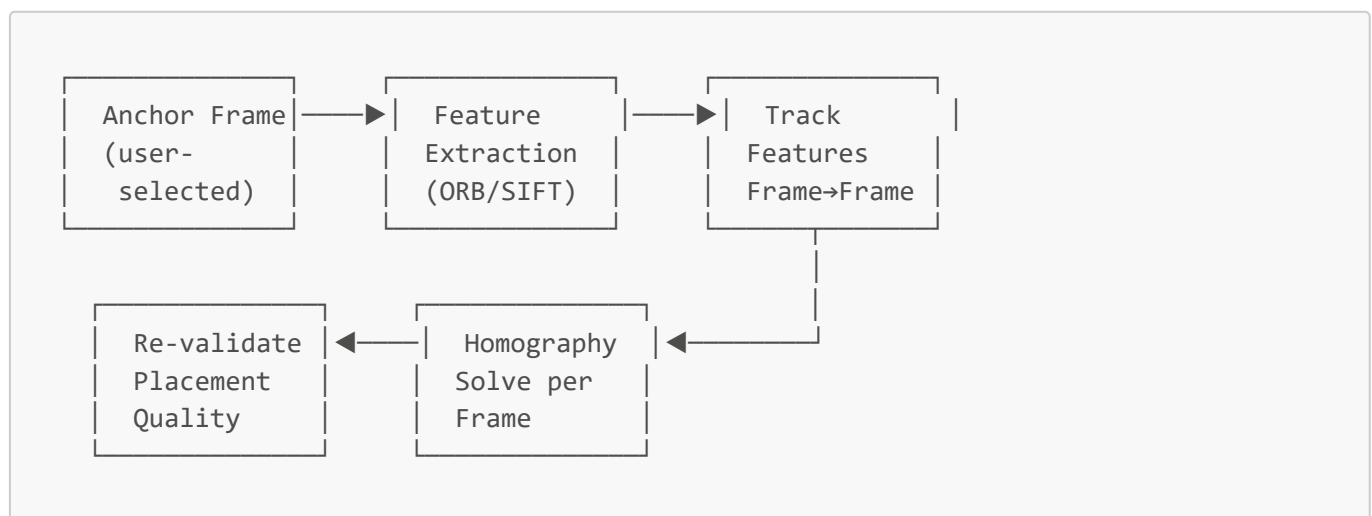
- $C_{confidence}$ = CV model confidence (0–1)
- $V_{visibility}$ = Surface prominence in frame (0–1)
- $D_{duration}$ = Seconds surface is visible
- S_{size} = Relative screen area (%)
- $O_{occlusion}$ = Occlusion penalty
- $P_{brand_conflict}$ = Competitive separation penalty (conflicting brand presence)

Brand-Safety Pipeline (MReq 4):

1. Check surface against permanent exclusion categories → **auto-reject**
2. Detect existing brands in scene (logo/text/category) → flag conflicts
3. Human approval mandatory → **no output without approval**

7.4 Motion Tracking Subsystem

Purpose: Lock a brand asset to a surface across camera movement (pan, tilt, zoom, rotation).

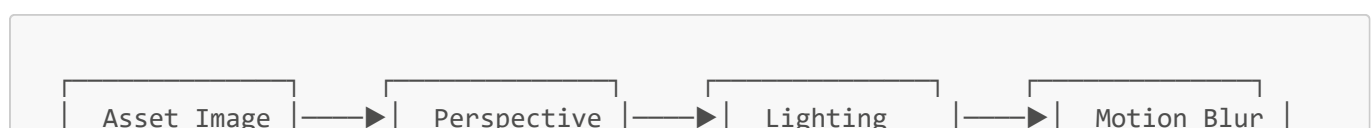


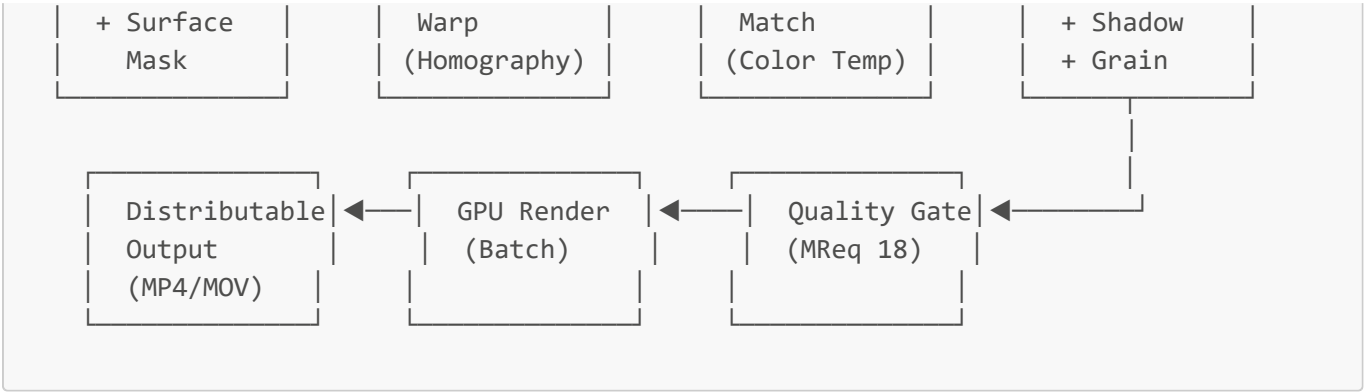
Tracking Methods:

1. **Primary:** OpenCV planar tracker (homography-based)
2. **Fallback:** Point tracker (KLT / optical flow) for complex shots
3. **Post-track QA:** Re-validate placement after tracking; flag drift/slip

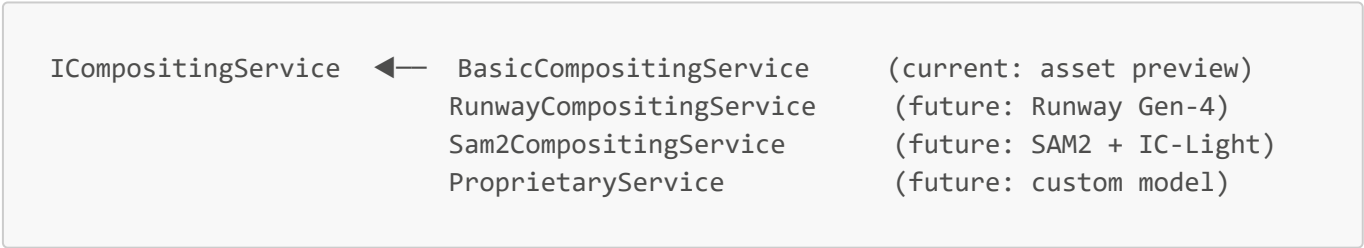
7.5 Compositing & Rendering Subsystem

Purpose: Warp, relight, blur, and shadow-match brand assets into video; render distributable output on GPU.





Compositing Service Strategy (Swappable):

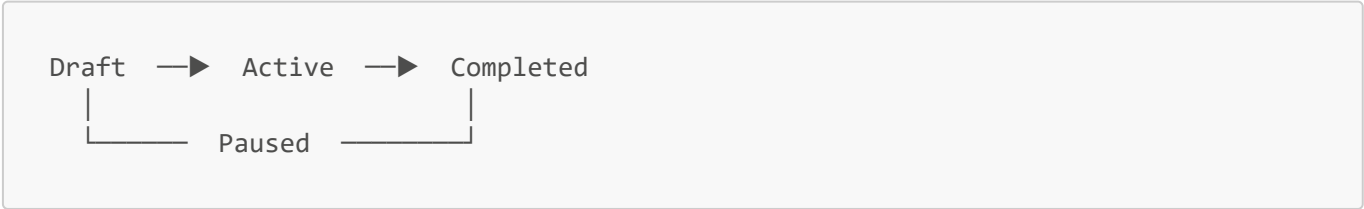


Export Presets: Broadcast (ProRes), Streaming (H.264), Social (Vertical 9:16 optional)

7.6 Campaign & Inventory Subsystem

Purpose: Manage advertisers, campaigns, creative assets, scheduling, and regional targeting.

Campaign Lifecycle:

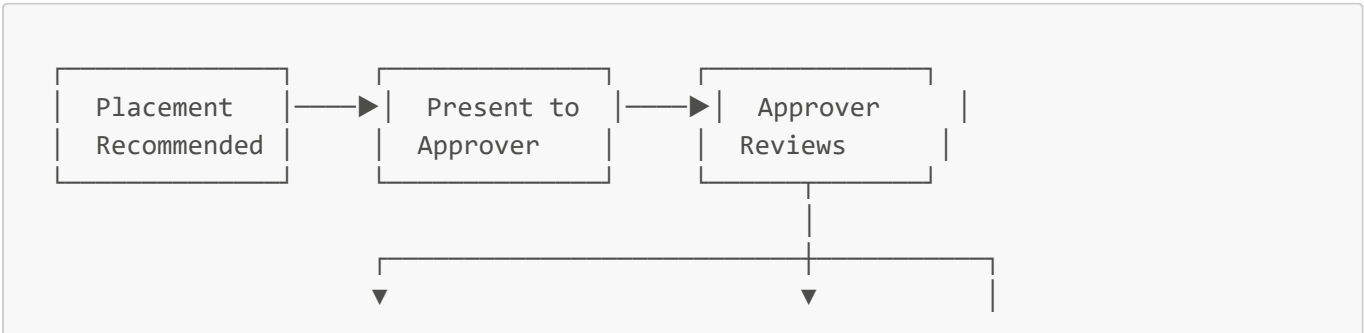


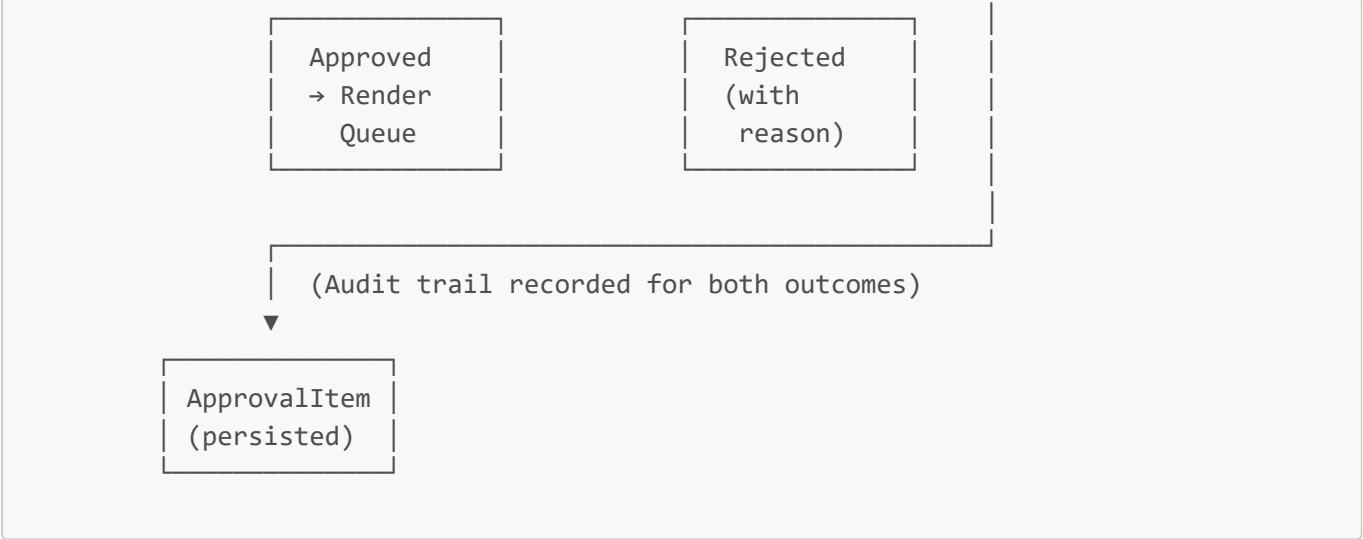
Key Rules:

- Campaign naming follows strict structure: `XX00XX00_XXXX XX00XX00_XXXX`
- Assets categorized per client, brand, dimension
- Regional targeting: same content → different brand per market
- Competitive separation algorithm prevents conflicting brand placements

7.7 Approval Workflow Subsystem

Purpose: Mandatory human-in-the-loop approval for every placement; permanent audit trail.





7.8 Analytics & BI Subsystem

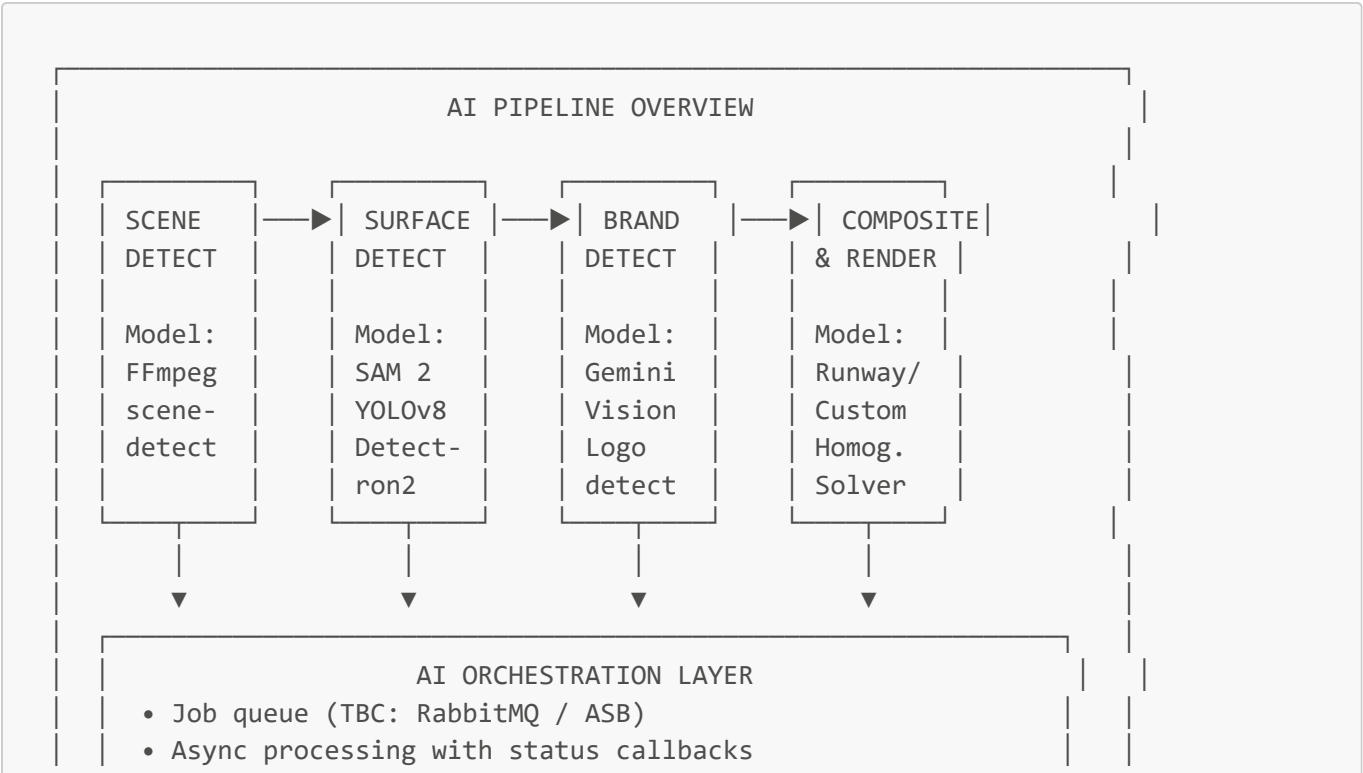
Metrics tracked (MReq 19):

- Content items ingested, analysed, rendered
- Ad slots created and assigned
- Estimated impressions and exposure duration per placement
- Peak and average processing/render times
- Revenue vs. render cost (cost control)

8. AI / ML Pipeline Design

⚠ Status: AI tools for segmentation, tracking, and compositing are **yet to be confirmed**. The architecture below uses interface abstractions so any engine can be swapped without re-architecting.

8.1 AI Integration Points



- Retry with exponential backoff
- Model version tracking per scene
- Cost tracking per AI call

8.2 AI Model Evaluation Framework

Each AI function will be evaluated against these criteria before final selection:

Criterion	Weight	Description
Accuracy	High	Detection precision/recall; compositing realism
Speed	High	Inference time per frame; GPU utilization
Cost	Medium	Per-minute API cost vs. inventory value earned
Integration Complexity	Medium	API maturity, SDK availability, containerization ease
Data Sovereignty	High	On-premise option; data stays in region
Vendor Lock-in Risk	Medium	Open-source vs. proprietary; migration cost

8.3 AI Service Abstractions (Implemented)

```
// Already in codebase – allows swapping AI engines
public interface ICompositingService
{
    Task<CompositedFrame> CompositeAsync(CompositingRequest request);
}

// Proposed additional abstractions:
public interface ISceneDetectionService { ... }
public interface ISurfaceDetectionService { ... }
public interface IMotionTrackingService { ... }
public interface IBrandDetectionService { ... }
```

8.4 Current AI Integration: Google Gemini

The frontend currently integrates with Google Gemini ([@google/genai](#) v2.4) for generative AI capabilities. The scope (scene description, brand recommendations, prompt generation) is to be confirmed against the production AI pipeline.

9. API Design

9.1 API Conventions

- **Base URL:** [/api](#)
- **Protocol:** HTTPS (enforced)

- **Auth:** JWT Bearer token in **Authorization** header
- **Content-Type:** **application/json**
- **Naming:** camelCase JSON properties
- **Versioning:** Embedded in route (future: **/api/v1/...**)

9.2 Endpoint Inventory

Authentication

Method	Endpoint	Auth	Description
POST	/api/auth/login	No	Authenticate user, return JWT
POST	/api/auth/register	No	Register new user

Content & Scenes

Method	Endpoint	Auth	Description
GET	/api/content	Yes	List all content items
GET	/api/content/{id}	Yes	Get content detail
POST	/api/content/upload	Yes	Upload new content
GET	/api/content/{id}/scenes	Yes	List scenes for content
GET	/api/scenes/{id}/surfaces	Yes	List surfaces for scene
PUT	/api/scenes/{id}	Yes	Update scene QA status / AI prompt

Surfaces & Placements

Method	Endpoint	Auth	Description
GET	/api/surfaces	Yes	List all surfaces (with filters)
POST	/api/surfaces	Yes	Create surface placement
PUT	/api/surfaces/{id}	Yes	Update surface status/approval

Campaigns & Assets

Method	Endpoint	Auth	Description
GET	/api/campaigns	Yes	List campaigns
POST	/api/campaigns	Yes	Create campaign
DELETE	/api/campaigns/{id}	Yes	Delete campaign
GET	/api/campaigns/{id}/assets	Yes	Get campaign + its assets
GET	/api/assets	Yes	List creative assets

Method	Endpoint	Auth	Description
POST	/api/assets	Yes	Upload creative asset

Compositing & Renders

Method	Endpoint	Auth	Description
POST	/api/compositing/preview	Yes	Generate composited preview frame
GET	/api/renders	Yes	List render jobs
POST	/api/renders	Yes	Submit render job
GET	/api/renders/{id}/status	Yes	Check render progress

Operations

Method	Endpoint	Auth	Description
GET	/api/logs	Yes	Query event logs
GET	/api/alarms	Yes	List alarms
PUT	/api/alarms/{id}	Yes	Acknowledge/clear alarm
GET	/api/users	Admin	List users
POST	/api/users	Admin	Create user

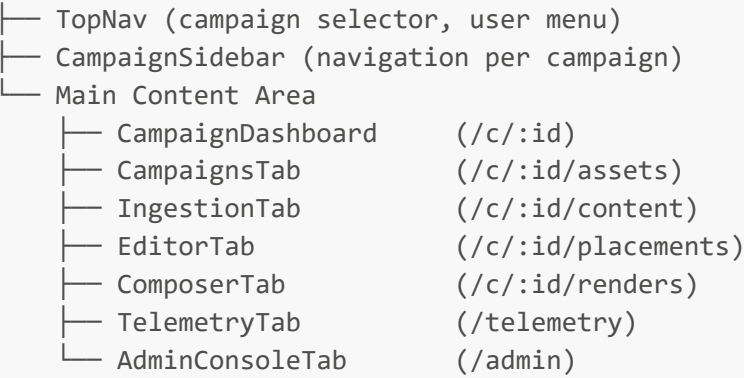
9.3 Standard Response Envelope

```
{
  "data": { ... },
  "error": null,
  "meta": {
    "page": 1,
    "pageSize": 20,
    "totalCount": 150
  }
}
```

10. Frontend Architecture

10.1 Component Tree

```
<BrowserRouter>
  <App>
    ├── LoginScreen (unauthenticated)
    └── AuthenticatedApp
```



10.2 URL-Driven State

The application uses React Router with URL-derived state for full link sharing and browser back/forward support:

URL Pattern	View
/	Landing page
/c/:campaignId	Campaign dashboard
/c/:campaignId/assets	Asset library
/c/:campaignId/content	Content ingestion
/c/:campaignId/placements	Scene editor / surface QA
/c/:campaignId/renders	Compositing & renders
/c/:campaignId/reports	Analytics & reports
/admin	Admin console
/telemetry	System telemetry

10.3 State Management

- **Server State:** Fetched via `apiClient.ts` on component mount; no global cache (yet)
- **Auth State:** JWT token in `localStorage`; user session in memory
- **URL State:** Campaign ID and active view encoded in URL path (single source of truth)
- **Future:** Consider TanStack Query (React Query) for server-state caching, deduplication, and optimistic updates

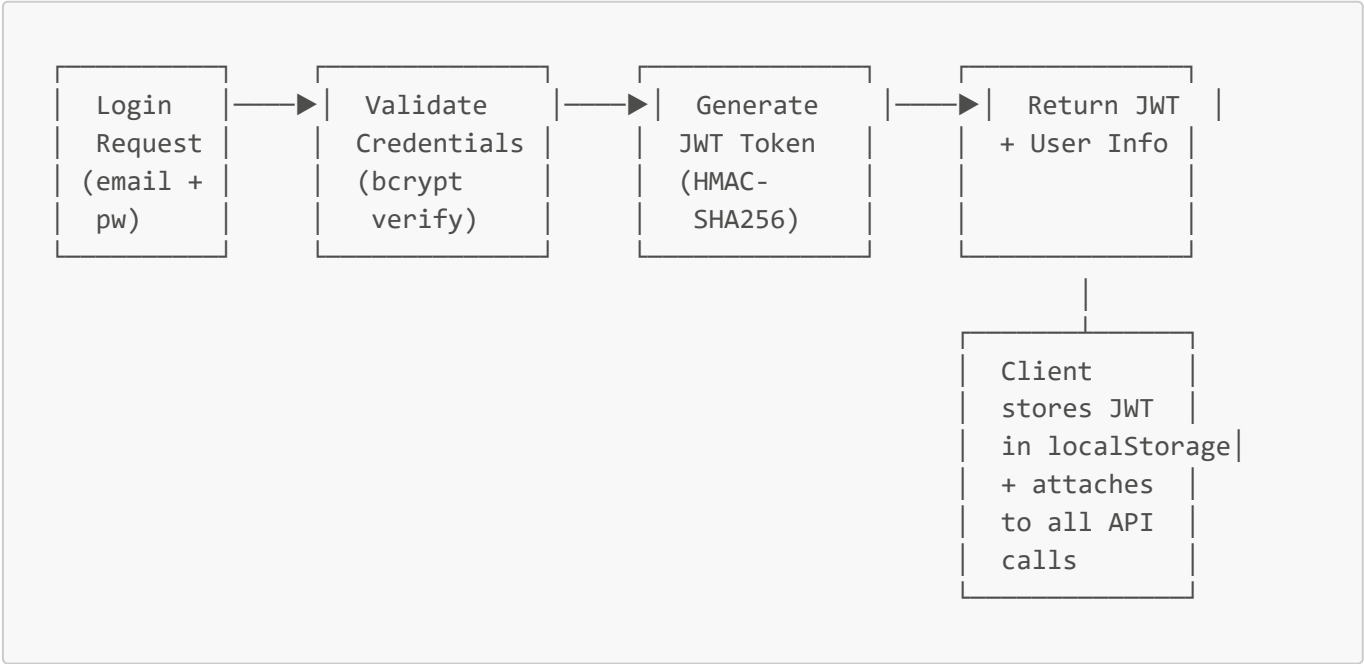
10.4 Key Libraries

Library	Usage
<code>react-router-dom</code> v7	Client-side routing, URL-driven state
<code>motion</code> (Framer Motion)	Page transitions, micro-interactions
<code>lucide-react</code>	Consistent icon set

Library	Usage
<code>tailwindcss</code> v4	Utility-first CSS with <code>@tailwindcss/vite</code> plugin
<code>@google/genai</code>	Gemini AI integration (client-side)

11. Security Architecture

11.1 Authentication Flow



11.2 Authorization Model (RBAC)

Role	Permissions
Admin	Full system access: user management, config, exclusion list, all data
Editor / Approver	Review scenes, approve/reject placements, manage content QA
Advertiser	Create campaigns, upload assets, view own campaigns and reports
Content Owner	Ingest content, view analysis results, approve placements on own content

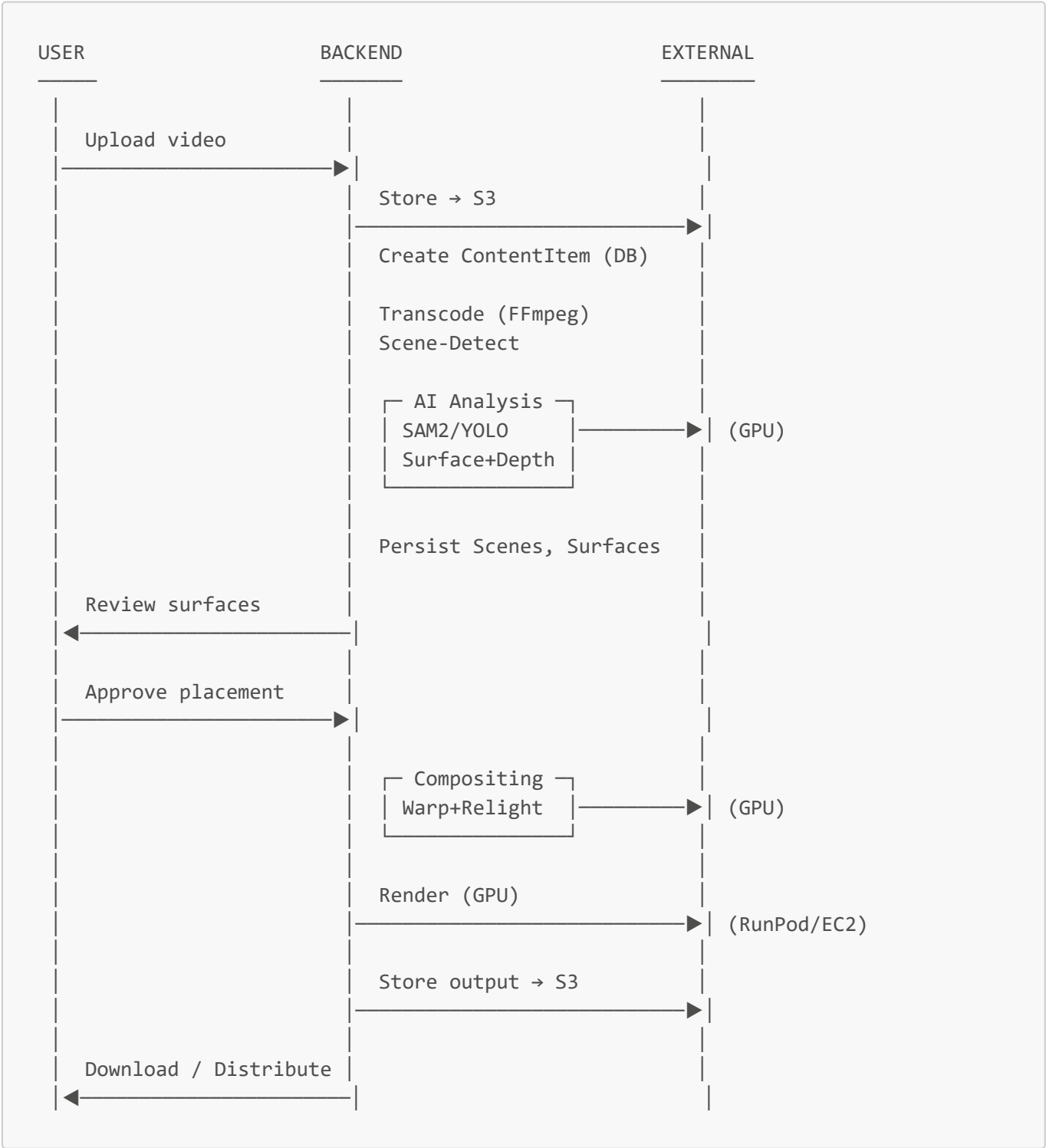
11.3 Security Measures

Measure	Implementation
Transport Security	HTTPS enforced
Password Storage	bcrypt hashing
Token Security	JWT with HMAC-SHA256; configurable expiry
CORS	Whitelist: <code>localhost:3000</code> (dev), <code>*.run.app</code> (production)
Input Validation	ASP.NET model validation; all request fields validated

Measure	Implementation
SQL Injection	EF Core parameterized queries
Session Timeout	JWT expiry + auto-logout after inactivity (MReq 8)
Audit Trail	All approvals, user actions, and config changes logged

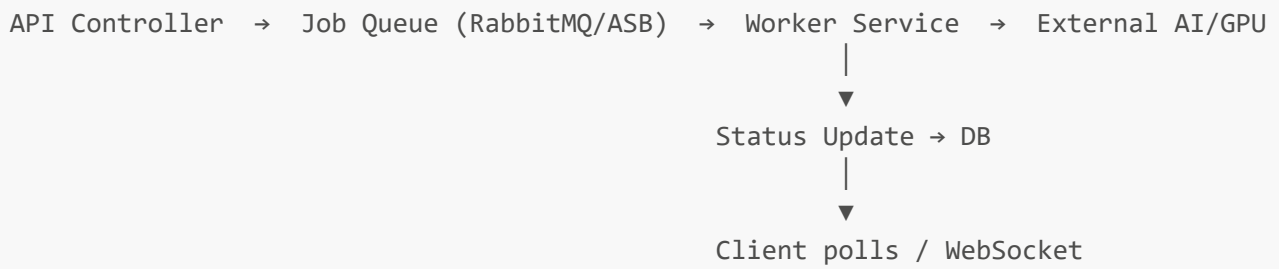
12. Data Flow & Integration Patterns

12.1 Content Processing Pipeline



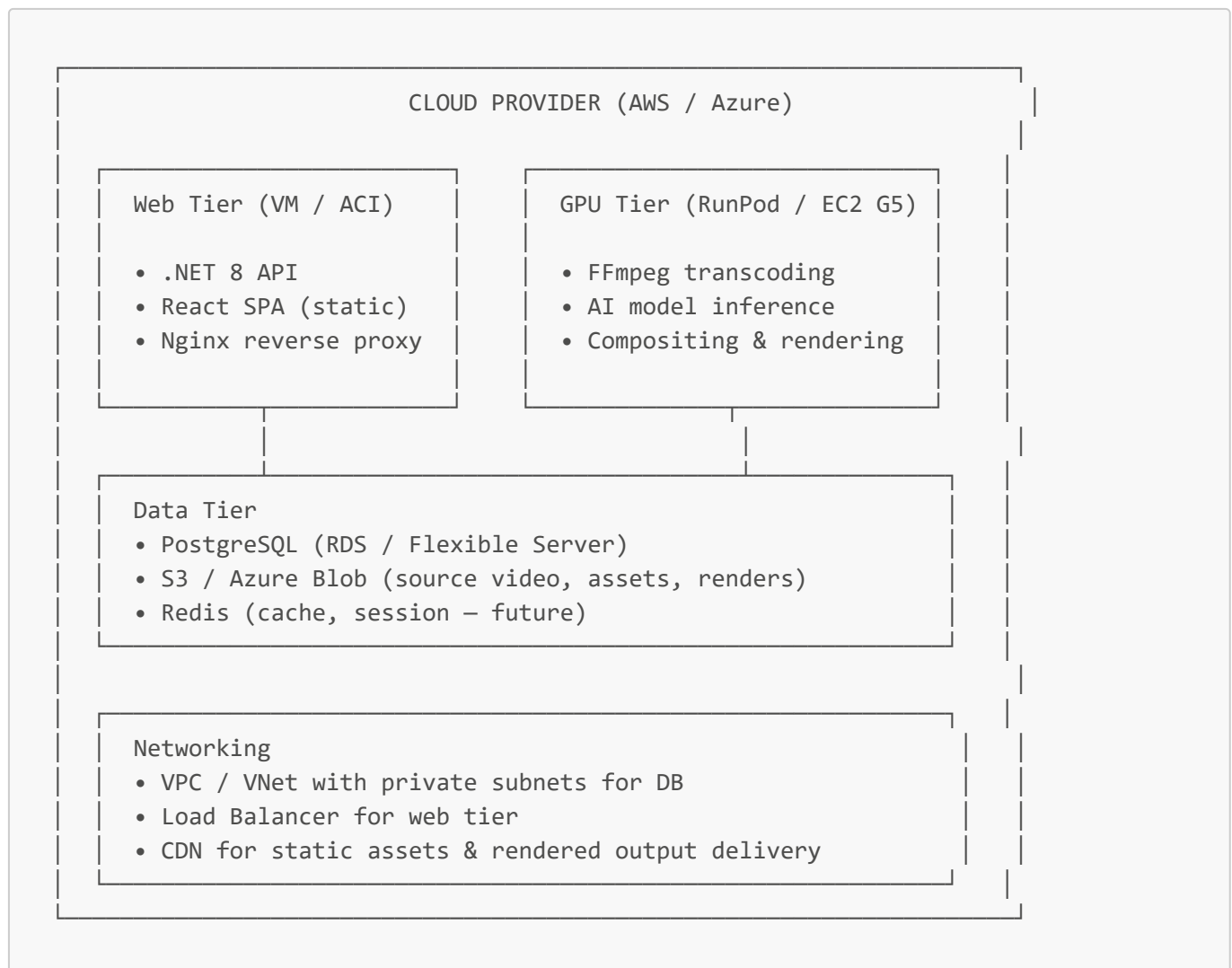
12.2 Async Job Pattern (Proposed)

For long-running operations (AI analysis, rendering), an async job queue is recommended:



13. Deployment Architecture

13.1 Target Environment



13.2 Containerization Strategy (Planned)

```

# Proposed: Multi-stage Docker build for .NET API
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY dotnet-api/ .
  
```

```
RUN dotnet publish -c Release -o /app

FROM mcr.microsoft.com/dotnet/aspnet:8.0
WORKDIR /app
COPY --from=build /app .
ENTRYPOINT ["dotnet", "Afrobotics.Bit.Api.dll"]
```

13.3 Infrastructure as Code (Recommended)

- **Terraform** or **Bicep** for cloud resource provisioning
- **GitHub Actions** for CI/CD pipeline
- Environment-specific config via **.env** / appsettings

14. Monitoring & Observability

14.1 Logging (MReq 20)

- **Event Logs** persisted to **EventLog** table via **ILogService**
- Structure: **timestamp** | **eventCode** | **severity** | **module** | **user** | **description**
- Severities: **Info**, **Warning**, **Major**, **Critical**

14.2 Alarms (MReq 21)

- Generated automatically from critical events
- Cleared automatically when condition resolves
- Target conditions: unavailable storage, GPU service, database, notification service
- Persisted to **AlarmItem** table; visible in Telemetry dashboard

14.3 Usage Records (MReq 22)

- Record all successful/failed API requests
- Periodic collation and archival to external storage

14.4 BI Dashboard (MReq 19)

Real-time metrics in admin/telemetry views:

- Content processed today
- Active campaigns
- Render queue depth
- System health (storage, GPU, DB status)

14.5 Recommended Additional Tooling

Tool	Purpose
Application Insights / CloudWatch	APM, distributed tracing
Serilog / NLog	Structured logging

Tool	Purpose
Prometheus + Grafana	Infrastructure metrics, custom business metrics
Sentry / Raygun	Error tracking and alerting
Health Checks	ASP.NET Core Health Checks endpoint for load balancer probes

15. Non-Functional Requirements

15.1 Performance

Metric	Target
API response time (p95)	< 500ms (reads), < 2s (writes)
Content ingestion (per minute)	[TBC during sizing]
Concurrent render jobs	[TBC during sizing]
Render cost per minute	Must be below inventory value earned

15.2 Availability

- Core services deployed redundantly (MReq 17)
- Single node failure shall not stop processing
- In-progress render jobs recoverable/re-queueable
- Target: 99.9% uptime for web tier

15.3 Scalability

- Stateless web tier: scale horizontally behind load balancer
- GPU tier: scale via job queue depth
- Database: PostgreSQL read replicas for reporting queries

15.4 Data Integrity

- Source content duration immutable (MReq 1)
- Approval audit trail permanent (MReq 4, 11)
- Brand-safety exclusion list only additive (MReq 4)

16. Risks, Assumptions & Mitigations

16.1 Key Risks

Risk	Impact	Likelihood	Mitigation
AI model accuracy insufficient	Poor surface detection, unrealistic compositing	Medium	Interface abstraction allows swapping; human approval gate catches errors

Risk	Impact	Likelihood	Mitigation
GPU compute cost exceeds budget	Unit economics unviable	Medium	Cost tracking per job; cost-vs-revenue monitoring; batch optimization
Vendor lock-in to AI provider	Migration cost, pricing changes	Medium	All AI behind interfaces; prioritize open-source models (SAM2, OpenCV)
Content rights / licensing issues	Legal exposure	Low	Content owner approval workflow; regional targeting respects rights
Real-time compositing performance	Slow pipeline, poor UX	Medium	Async job architecture; progressive preview generation
Data sovereignty (African markets)	Regulatory compliance	Medium	On-premise deployment option; local cloud regions (Azure SA, AWS Cape Town)

16.2 Key Assumptions

- 1. Source content is provided in standard broadcast formats (MP4, MOV, MXF)
- 2. GPU compute is available on-demand (RunPod, EC2, or on-prem)
- 3. Advertisers will provide brand assets in suitable formats (PNG, SVG, MOV with alpha)
- 4. Human approvers are available to review placements before rendering
- 5. Broadcaster/content-owner integration APIs are available (MReq 16)

17. Appendix

17.1 Glossary

Term	Definition
VPP	Virtual Product Placement
BIT	Brand Insertion Technology
CV	Computer Vision
SAM	Segment Anything Model (Meta)
DSP	Demand-Side Platform
CPM	Cost per Mille (cost per thousand impressions)
RBAC	Role-Based Access Control
MReq	Mandatory Requirement (numbered, testable)
Homography	Perspective transformation matrix mapping one plane to another

17.2 File Structure

```

BIT-APP/
├── .github/
│   └── requirements.txt                # Original MReq specification
├── docs/
│   └── DESIGN_DOCUMENT.md            # This document
├── dotnet-api/                       # .NET 8 Backend
│   ├── Controllers/                 # API endpoints
│   ├── Data/                        # EF Core DbContext, Seeder
│   ├── DTOs/                       # Request/response objects
│   ├── Migrations/                 # EF Core database migrations
│   ├── Models/                     # Domain entities
│   ├── Repositories/               # Data access layer
│   ├── Services/                   # Business logic
│   ├── Uploads/                    # Local file storage (dev)
│   └── Program.cs                   # App entry point, DI, middleware
├── dotnet-api.Tests/                 # xUnit test project
├── src/                              # React 19 Frontend
│   ├── components/                  # UI components
│   │   ├── CampaignsTab.tsx
│   │   ├── IngestionTab.tsx
│   │   ├── EditorTab.tsx
│   │   ├── ComposerTab.tsx
│   │   ├── TelemetryTab.tsx
│   │   ├── AdminConsoleTab.tsx
│   │   ├── CampaignDashboard.tsx
│   │   ├── CampaignSelector.tsx
│   │   └── CampaignSidebar.tsx
│   ├── apiClient.ts                 # Centralized API client
│   ├── App.tsx                      # Root component with routing
│   ├── main.tsx                     # Entry point
│   ├── types.ts                     # TypeScript interfaces
│   └── document.ts                   # Static documentation content
├── package.json                     # Frontend dependencies
├── vite.config.ts                    # Vite build config + API proxy
├── tsconfig.json                     # TypeScript config
└── README.md                         # Quick-start guide

```

17.3 References

- [Mirriad — Reference Product](#)
- [SAM 2 — Segment Anything Model](#)
- [Runway Gen-4](#)
- [OpenCV — Planar Tracking](#)
- [EF Core with PostgreSQL](#)

End of Design Document